

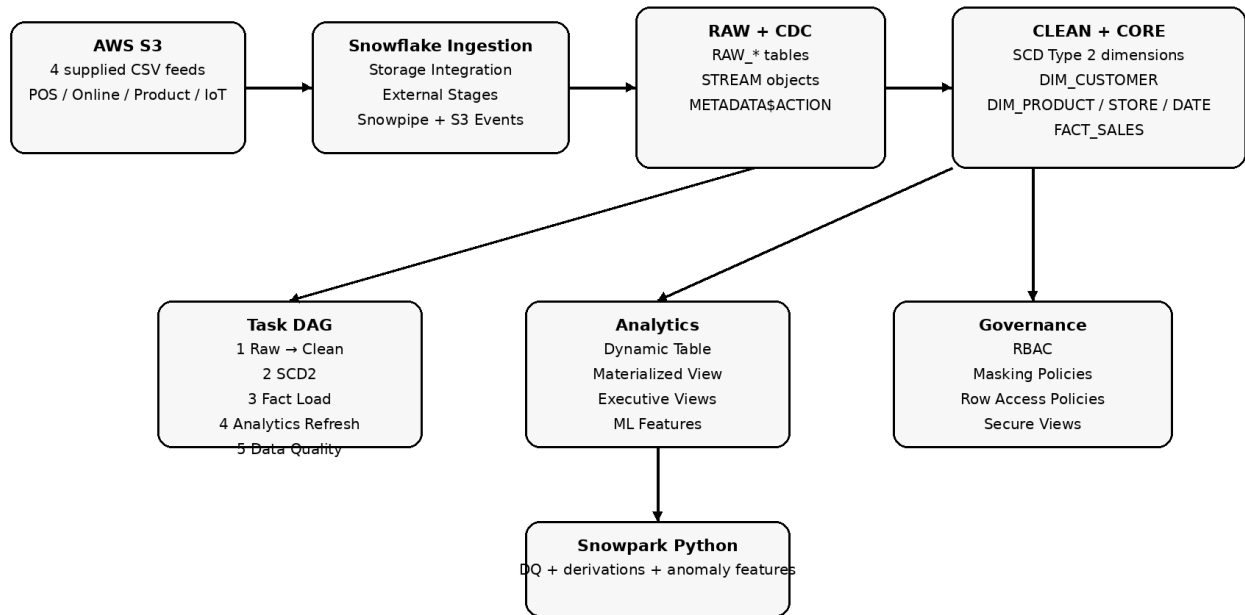
GLOBRORETAIL

Real-Time Retail Sales Analytics Platform

Research-Style Technical Capstone Report

Snowflake + AWS S3 + Snowpipe + Streams + Tasks + Snowpark Python

GloboRetail — Real-Time Snowflake Architecture



Real-Time Retail Sales Analytics Platform Using Snowflake & AWS

Research-Style Technical Project Report

Project: GloboRetail Real-Time Retail Sales Analytics Platform

Platform: Snowflake + AWS S3 + Snowpipe + Streams + Tasks + Snowpark Python

Dataset: Supplied GloboRetail capstone datasets

Report type: Technical capstone / research-style implementation report

Abstract

This project implements an end-to-end retail analytics platform for GloboRetail using Snowflake and Amazon Web Services. The design addresses multi-source ingestion, continuous loading, change data capture, task-based orchestration, dimensional modeling, SCD Type 2 history, data quality, Snowpark transformations, data masking, row-level security, performance optimization, cost governance and Time Travel recovery. The implementation is grounded in the supplied capstone specification, supplied star-schema DDL and four supplied CSV datasets containing 1,000 records each.

The source data consists of point-of-sale sales, online orders, product catalog updates and inventory device logs. The architecture separates raw, clean, core, analytics, security and operations layers. AWS S3 is used as the landing zone. Snowflake Storage Integration and external stages provide credential delegation, while Snowpipe provides event-driven ingestion. Snowflake Streams capture incremental changes and a task DAG orchestrates downstream processing. The core model preserves the required DIM_CUSTOMER, DIM_PRODUCT, DIM_STORE, DIM_DATE and FACT_SALES names and extends them with operational and governance attributes.

A key research and engineering constraint is that several required business attributes are absent from the supplied files. POS records contain no transaction date, and the sources contain no customer PII, store descriptive attributes, discount rate or tax rate. The implementation therefore uses explicit, deterministic demo enrichments and configurable assumptions while documenting them as synthetic rather than real business facts. This prevents accidental misrepresentation of the source data while still demonstrating the required Snowflake capabilities.

Keywords: Snowflake, AWS S3, Snowpipe, Streams, Tasks, CDC, SCD Type 2, Snowpark, Data Masking, Row Access Policy, RBAC, Dynamic Tables, Materialized Views, Time Travel, Retail Analytics.

1. Introduction

GloboRetail is described as a multinational retailer operating across multiple countries and receiving high-volume data from POS terminals, online channels, inventory devices and third-party sources. The supplied problem statement identifies long batch-processing windows, limited real-time visibility, manual pipelines, excessive exposure of sensitive customer information and inefficient warehouse usage as the main business problems.

The proposed solution is a cloud-native Snowflake data platform. The architecture moves from source ingestion to raw storage, CDC, cleansing, dimensional modeling, analytics and governance. Snowflake is used as the central analytical platform so that transformations and feature engineering can occur close to the data rather than requiring repeated external exports.

2. Problem Statement

The platform must support:

1. AWS S3-based file ingestion.
2. Snowpipe continuous ingestion.
3. Streams for inserts, updates and deletes.

4. A task DAG for transformation and orchestration.
5. SQL and Snowpark transformations.
6. Star-schema dimensions and facts.
7. SCD Type 2 history.
8. Data quality and logging.
9. Dynamic data masking and secure views.
10. Row-level region restrictions.
11. RBAC and least privilege.
12. Stored procedures for validation, SCD2 and recovery.
13. Clustering and query-pruning demonstrations.
14. Materialized/dynamic analytical objects.
15. Warehouse auto-suspend/auto-resume and resource monitoring.
16. Query-history analysis.
17. Time Travel recovery.
18. Optional Snowpark ML feature engineering.

3. Source Dataset Analysis

The supplied dataset package contains four CSV files with 1,000 records each.

3.1 POS Sales

Columns: sale_id, customer_id, product_id, store_id, sale_amount.

Observed values:

- 1,000 rows.
- 617 distinct customers.
- 635 distinct products.
- 100 stores.
- Sale amount range: 10.55 to 499.68.
- Average sale amount: 248.3763.
- Total supplied POS sales amount: 248,376.30.
- No nulls.
- No duplicate full rows.

3.2 Online Orders

Columns: order_id, customer_id, product_id, order_date, order_amount.

Observed values:

- 1,000 rows.
- 623 distinct customers.
- 618 distinct products.
- Order dates from 2024-01-01 to 2024-12-30.
- Order amount range: 20.56 to 799.08.
- Average order amount: 411.17199.
- Total supplied online order amount: 411,171.99.

- No nulls.
- No duplicate full rows.

3.3 Product Catalog

Columns: product_id, product_name, category, price, status.

Observed values:

- 1,000 products.
- Price range: 5.53 to 997.67.
- Average price: 499.13744.
- Categories: Grocery 265, Clothing 251, Home 244, Electronics 240.
- Status: ACTIVE 513, DISCONTINUED 487.
- No nulls.
- No duplicate full rows.

3.4 Inventory Device Logs

Columns: device_id, store_id, product_id, quantity_on_hand, log_timestamp.

Observed values:

- 1,000 rows.
- 1,000 distinct devices.
- 100 stores.
- Quantity range: 0 to 500.
- Average quantity on hand: 252.435.
- Dates from 2024-01-01 to 2024-12-30.
- No nulls.
- No duplicate full rows.

3.5 Cross-source integrity

All product IDs observed in POS, online and inventory sources exist in the product catalog. POS and inventory share the same 100-store identifier universe. POS and online customers overlap on 386 IDs, providing a natural conformed customer dimension.

4. Data-Modeling Strategy

The supplied star-schema names are retained exactly. The implementation extends them where required for operational processing.

4.1 Dimension tables

DIM_CUSTOMER contains the required customer key and business identifier plus SCD2 metadata. DIM_PRODUCT preserves product attributes and adds status/history. DIM_STORE contains the required store attributes plus country and region. DIM_DATE provides a 2024 calendar.

4.2 Fact table

FACT_SALES retains the supplied required keys and SALES_AMOUNT. Additional measures include DISCOUNT_AMOUNT, TAX_AMOUNT and REVENUE_AMOUNT, together with source type, source transaction ID and timestamps.

4.3 Inventory support

FACT_INVENTORY is added because the capstone explicitly includes IoT inventory device logs. This table also supports low-stock feature engineering.

5. Source-Data Constraints and Controlled Assumptions

The implementation does not silently invent source facts.

5.1 POS transaction date

pos_sales.csv has no transaction date. Therefore, LOADED_AT is used as the POS TRANSACTION_DATE proxy. This is suitable for a pipeline demonstration but should be replaced with an actual source event timestamp in production.

5.2 Customer PII

The source contains no email, phone or payment-card data. Synthetic deterministic values are generated solely so dynamic masking can be demonstrated. They are explicitly marked as demo enrichment.

5.3 Store attributes

The source provides STORE_ID only. Store name, city, state, country and region are deterministic demo attributes. A production system should source these from a master store system.

5.4 Tax and discount

The source contains no tax or discount fields. The demo uses 18% tax and 0% discount through the OPS.PIPELINE_PARAMETERS table. These are configuration assumptions, not source facts.

6. System Architecture

The platform follows the sequence:

AWS S3 -> Storage Integration -> External Stages -> Snowpipe -> RAW -> Streams -> CLEAN -> Task DAG -> CORE Star Schema -> Analytics -> Governance

Snowpark Python operates as the in-platform transformation and feature-engineering layer. Operations tables maintain pipeline and data-quality logs.

6.1 Architectural layers

Landing: AWS S3.

Ingestion: Storage Integration, external stages and Snowpipe.

Raw: Source-aligned ingestion tables.

CDC: Streams.

Clean: Source-aligned cleaned copies.

Core: Star schema and inventory facts.

Analytics: Dynamic table, materialized view and executive views.

Security: Masking, row access policies and secure views.

Operations: Pipeline logs, DQ logs and configuration parameters.

7. Ingestion Pipeline

The project uses Snowflake Storage Integration rather than hard-coded AWS credentials. The storage integration delegates cloud access through a Snowflake-generated AWS identity. The S3 stages reference the integration and restrict the allowed bucket prefix.

Four Snowpipes are created:

- `PIPE\_POS\_SALES`
- `PIPE\_ONLINE\_ORDERS`

- `PIPE\_PRODUCT\_CATALOG`
- `PIPE\_INVENTORY\_LOGS`

Each pipe maps the CSV fields into the corresponding raw table and captures METADATA\$FILENAME and CURRENT_TIMESTAMP() for ingestion observability.

For auto-ingest, S3 ObjectCreate events are connected to the Snowflake-provided SQS notification channel. The system then loads newly arriving files without requiring a scheduled batch job.

8. Change Data Capture

Streams are created on all four raw tables. The streams expose Snowflake change metadata and allow downstream tasks to consume incremental records.

The pipeline uses:

RAW -> STREAM -> CLEAN -> CORE

The root task checks stream availability using SYSTEM\$STREAM_HAS_DATA. This avoids running the full pipeline when no source changes are waiting.

9. Task DAG

The orchestration graph is:

TASK_01_RAW_TO_CLEAN

↓

TASK_02_SCD2_DIMENSIONS

↓

TASK_03_LOAD_FACTS

↓

TASK_04_REFRESH_ANALYTICS

↓

TASK_05_DATA_QUALITY

The first task is event-triggered by stream data. Child tasks use AFTER dependencies. The final task runs data-quality checks and writes results to OPS.DATA_QUALITY_LOG.

The task design intentionally keeps deployment separate from activation. Tasks are created suspended and only resumed by 08_tasks/99_start_pipeline.sql after all dependencies have been deployed.

10. SCD Type 2

The customer dimension uses:

- `EFFECTIVE\_FROM`
- `EFFECTIVE\_TO`
- `IS\_CURRENT`
- `RECORD\_HASH`

The stored procedure SP_MERGE_CUSTOMER_SCD2 detects changes using a record hash, closes the current version and inserts a new version. This preserves historical dimension states instead of overwriting them.

The supplied static customer IDs do not contain natural changing customer attributes, so a live SCD2 change cannot be inferred from the source files alone. The implementation therefore provides the complete mechanism and uses deterministic attributes as the demonstration basis.

11. Stored Procedures

The implementation contains procedures for:

1. Pipeline logging.
2. Customer SCD2 processing.
3. Data-quality validation.
4. Analytics/feature refresh.
5. Failed-file reprocessing template.
6. Notification-email template.

JavaScript procedures use try/catch and return status messages. Snowflake Scripting is used for the optional email procedure with EXCEPTION handling.

12. Data Quality

The quality framework validates:

- Source row counts.
- Negative sales amounts.
- Negative online order amounts.
- Negative product prices.
- Negative inventory quantities.
- Unknown product references.
- Null revenue in the fact.
- Null product keys in downstream validation.

The supplied data passes the basic source checks because it contains no nulls, no negative numeric values and no product-reference failures.

13. Security Architecture

The project defines the required roles:

- `DATA\_ENGINEER`
- `DATA\_ANALYST`
- `DATA\_SCIENTIST`
- `BI\_ANALYST`
- `SECURITY\_ADMIN`

Dynamic masking policies protect:

- Customer email.
- Phone number.
- Payment card hash.

A row access policy restricts store visibility by region. In the demonstration configuration, BI analysts see North, data scientists see North/South, data engineers and security administrators see all regions, and general analysts receive broader access.

Secure views provide a controlled analytical interface without exposing direct raw access.

14. Snowpark Python

Snowpark is used for null handling, derivations, data-quality checks and feature engineering. The pipeline produces store-level revenue features including:

- Total revenue.
- Transaction count.
- Average transaction value.
- Revenue lag.
- Mean and standard deviation.
- Anomaly score.
- Low-stock support.

The main advantage is that transformations are executed in Snowflake rather than requiring full datasets to be moved to a local machine.

15. Analytics Layer

The project contains:

DT_DAILY_STORE_REVENUE — dynamic table with a 10-minute target lag.

MV_DAILY_REVENUE — materialized view for daily/source-level revenue reporting.

V_EXECUTIVE_SALES — simple executive reporting view.

The task-driven refresh procedure also manually refreshes the dynamic table after fact processing.

16. Performance Optimization

The fact table is clustered by:

(TRANSACTION_DATE, STORE_KEY)

This supports selective date/store predicates and demonstrates micro-partition pruning.

A concurrency warehouse is configured with one to three clusters and auto-suspend/auto-resume. A resource monitor is configured with a monthly credit quota and notification/suspend thresholds.

Query-history analysis records elapsed time, bytes scanned, partitions scanned and partitions total.

Important experimental limitation

The supplied data contains only 4,000 source rows. This is insufficient to make statistically meaningful production-scale clustering claims. The correct demonstration is to show the mechanism, inspect the query profile and state that measurable gains should be evaluated on representative production volumes.

17. Time Travel Recovery

The project demonstrates querying FACT_SALES at an earlier point using Time Travel and cloning the historical state into FACT_SALES_RECOVERY. This provides a simple disaster/recovery demonstration without an external duplicate dataset.

18. Cost Management

The project implements:

- XSMALL warehouses for the demonstration.
- 60-second auto-suspend.
- Auto-resume.
- A separate BI warehouse.
- A multi-cluster warehouse for concurrency spikes.
- A resource monitor.
- Dynamic-table target lag.
- Query-history monitoring.

The design avoids claiming a specific credit saving because actual cost depends on account pricing, cloud region, warehouse size, query frequency and workload concurrency.

19. Experimental Evaluation Plan

A production-scale evaluation should compare:

Metric	Baseline	Optimized
Query elapsed time	Capture before clustering	Capture after clustering
Bytes scanned	Capture	Capture
Partitions scanned	Capture	Capture
Partitions total	Capture	Capture
Pruning percentage	Calculate	Calculate
Warehouse credits	Capture	Capture
Concurrent query throughput	Capture	Capture

The project provides the SQL required to collect these measurements. Actual numbers must be captured from the user's Snowflake account rather than fabricated in advance.

20. End-to-End Execution Sequence

1. Create Snowflake database, schemas and warehouses.
2. Create file formats.
3. Create raw tables.
4. Create streams.
5. Create clean tables.
6. Create core dimensions/facts.
7. Configure AWS IAM.
8. Create Snowflake Storage Integration.
9. Create S3 stages.
10. Create Snowpipes.
11. Configure S3 notifications.
12. Upload supplied CSV files.
13. Seed store/date dimensions.
14. Deploy stored procedures.
15. Deploy analytics objects.

16. Deploy RBAC.
17. Deploy security policies.
18. Deploy task DAG.
19. Start the task DAG.
20. Validate counts and DQ.
21. Run Snowpark.
22. Run performance analysis.
23. Run Time Travel demonstration.
24. Capture screenshots and final metrics.

21. Demonstration Procedure

The recommended 15-minute demonstration begins with the source files and ends with business outcomes.

1. Source data.
2. S3 landing.
3. Snowpipe.
4. Raw tables.
5. Streams.
6. Task DAG.
7. Star schema.
8. Security.
9. Snowpark.
10. Performance.
11. Time Travel.
12. Business KPI summary.

The complete presenter script is stored in docs/DEMO_SCRIPT.md.

22. Reproducibility

The project is organized as numbered SQL and Python artifacts. All environment-specific values are isolated in configuration examples. No AWS secret key is embedded in SQL or Python source.

A fresh implementation should follow docs/EXECUTION_ORDER.md and docs/SETUP_STEPS.md exactly.

23. Discussion

The implementation demonstrates how a single Snowflake platform can combine ingestion, CDC, orchestration, analytical modeling and governance. The most important design principle is separation of responsibilities: S3 is the landing layer, Snowpipe handles continuous loading, streams expose changes, tasks orchestrate transformations, dimensions/facts provide analytical structure, Snowpark supports advanced processing, and security policies protect data at query time.

The source-data limitations also demonstrate a realistic engineering lesson: an architecture should not fabricate business facts merely to satisfy a target schema. Instead, missing attributes should be identified, sourced from the appropriate master system in production, or explicitly represented as controlled demo enrichments.

24. Conclusion

The completed capstone provides an executable reference architecture for a real-time retail analytics platform. It covers the requested Snowflake capabilities and preserves the source dataset and supplied star-schema naming conventions. The platform is designed for continuous ingestion, incremental processing, historical dimensions, secure analytics, Snowpark feature engineering, operational observability and cost-aware compute.

The next production step would be to replace the synthetic enrichment columns with governed master-data feeds, provide an actual POS event timestamp, tune clustering against production-scale data, validate security role mappings with the organization's IAM model and capture real performance/cost measurements from representative workloads.

25. Project Artifact Inventory

SQL

- `01\_setup/00\_database.sql`
- `01\_setup/01\_file\_formats.sql`
- `02\_ingestion/01\_storage\_integration.sql`
- `02\_ingestion/02\_stages.sql`
- `02\_ingestion/03\_pipes.sql`
- `02\_ingestion/04\_manual\_initial\_load.sql`
- `03\_raw/01\_raw\_tables.sql`
- `04\_cdc/01\_streams.sql`
- `04\_cdc/02\_clean\_tables.sql`
- `05\_transform/01\_core\_tables.sql`
- `05\_transform/02\_date\_store\_seed.sql`
- `05\_transform/03\_customer\_product\_merge.sql`
- `05\_transform/04\_fact\_load.sql`
- `06\_security/01\_roles\_and\_grants.sql`
- `06\_security/02\_policies.sql`
- `07\_procedures/01\_stored\_procedures.sql`
- `07\_procedures/02\_reprocess\_and\_alerts.sql`
- `08\_tasks/01\_task\_dag.sql`
- `08\_tasks/99\_start\_pipeline.sql`
- `10\_analytics/01\_dynamic\_table\_and\_views.sql`
- `11\_performance/01\_optimization.sql`
- `11\_performance/02\_query\_history.sql`
- `11\_performance/03\_time\_travel.sql`
- `13\_tests/01\_validation.sql`

Python

- `09\_snowpark/snowpark\_pipeline.py`
- `09\_snowpark/python\_worksheet\_version.py`

Documentation

- `README.md`

- ``docs/SETUP_STEPS.md``
- ``docs/EXECUTION_ORDER.md``
- ``docs/ARCHITECTURE.md``
- ``docs/DATA_MODEL.md``
- ``docs/DEMO_SCRIPT.md``
- ``docs/KNOWN_LIMITATIONS_AND_ENVIRONMENT_VALUES.md``

References

1. Snowflake Documentation — Automating Snowpipe for Amazon S3.
2. Snowflake Documentation — Creating S3 external stages.
3. Snowflake Documentation — Snowflake Tasks and task graphs.
4. Snowflake Documentation — Snowpark Developer Guide for Python.
5. Snowflake Documentation — Dynamic Data Masking.
6. Snowflake Documentation — Row Access Policies.
7. Snowflake Documentation — Dynamic Tables.
8. Supplied ``CAPSTONE_PROJECT.docx``.
9. Supplied ``CAPSTONE DATA SETS.docx``.
10. Supplied ``star_schema_tables.sql``.
11. Supplied ``pos_sales.csv``, ``online_orders.csv``, ``product_catalog.csv``, ``inventory_device_logs.csv``.